

08 · Did the loyalty rollout work? — difference-in-differences (CausalPy)

July 9, 2026

1 08 · Did the loyalty rollout work? — difference-in-differences (CausalPy)

Runs in the legacy environment (pymc<6): `make env-legacy, kernel cmp-legacy.`

The business decision. We launched the loyalty app in some stores, not others. Revenue in the launch stores went up — but revenue went up *everywhere* (a seasonal tide lifts all boats). Did the app add anything **net of that tide**, and is it worth rolling out to the whole chain?

1.0.1 The idea: two differences

Difference-in-differences (DiD) is the workhorse of quasi-experiments. It takes **two** differences and subtracts them:

$$\text{DiD} = \underbrace{(\bar{Y}_{\text{after}}^{\text{treated}} - \bar{Y}_{\text{before}}^{\text{treated}})}_{\text{treated change}} - \underbrace{(\bar{Y}_{\text{after}}^{\text{control}} - \bar{Y}_{\text{before}}^{\text{control}})}_{\text{control change}}.$$

The first difference (treated after — before) contains the app effect **plus** the seasonal tide; the second (control after — before) is the tide **alone**. Subtracting removes the tide and leaves the app effect. It’s the same “compare like with like” logic as everywhere else, applied *across time*.

1.0.2 The one assumption everything rests on: parallel trends

DiD is valid only if, **absent the app**, treated and control stores would have moved *in parallel* — the control’s change is a fair stand-in for what the treated stores’ change would have been. This is untestable after launch (we never see the treated stores’ no-app future), but the **pre-launch trends are the evidence**: if the two groups moved together *before* the app, parallel trends is credible. We check this formally with an **event study** (the effect period-by-period) and falsify with a **placebo** (a fake launch date in the pre-period, which should show ~ 0 effect).

On real data. Swap in your **own store/region panel** — one row per store per period with revenue, a treated/control flag, and a period index. The textbook public example is **Card & Krueger (1994)** on minimum wage and employment (fast-food restaurants in New Jersey vs Pennsylvania). One warning we *demonstrate live* in Step 7: if stores adopt at **different times** (“staggered rollout”), naive DiD can be badly biased and you need modern estimators (Callaway–Sant’Anna, Sun–Abraham).

7-step contract.

1.1 2 · Simulate a ground truth

40 stores, half get the loyalty app at week 12. Every store shares a seasonal pattern and has its own baseline; the app adds a **true €400/store/week**. Treated and control move in **parallel** before launch by construction — so DiD should recover €400, and the event study should show a flat pre-trend.

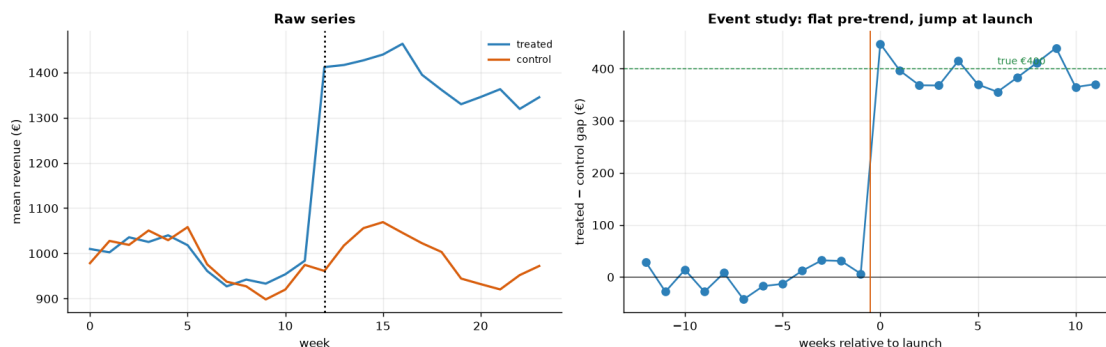
TRUE effect = €400/store/week · 40 stores, 24 weeks, launch week 12

	store	unit	t	week	group	post_treatment	post	revenue
0	store_00	0	0	0	1	False	0	1071.613014
1	store_00	0	1	1	1	False	0	1066.597066
2	store_00	0	2	2	1	False	0	1019.187212
3	store_00	0	3	3	1	False	0	834.246887
4	store_00	0	4	4	1	False	0	1030.278745

1.2 3 · Identify — the 2x2 estimand, parallel trends, and the event study

DiD estimates β_3 in $Y = \beta_0 + \beta_1 \text{group} + \beta_2 \text{post} + \beta_3 (\text{group} \times \text{post}) + \varepsilon$, which equals the difference of the two before/after differences. Under parallel trends, β_3 identifies the **ATT** — the average effect *on the treated (pilot) stores* in the post period; it need not equal what the other chain stores would see, which matters when we scale to 500 stores in Step 6. **Key assumption — parallel trends:** absent the app, treated and control revenue would have moved together. It's untestable post-launch, but the **event study** — the treated-minus-control gap in *each* period relative to launch — is the evidence: flat and ~0 before launch supports it; a jump at launch is the effect.

Pre-launch gaps average €22 (~0 supports parallel trends); post-launch gaps jump toward €400.



How to read the event study (right panel). This is the single most important plot in a DiD analysis. Each dot is the treated-minus-control revenue gap in one week, relative to launch (week 0). The story we *want* to see, and do: the dots are **flat and hovering around zero before launch** — that's the visual evidence for parallel trends, the assumption the whole method rests on — and then they **jump to ~ €400 at launch** and stay there. A rising or falling pre-launch

trend would have been a red flag that the two groups were already diverging, and we'd have had to abandon DiD for synthetic control (notebook 07). The left panel shows the raw series so you can see both groups riding the same seasonal wave.

1.3 4 · Estimate — Bayesian difference-in-differences

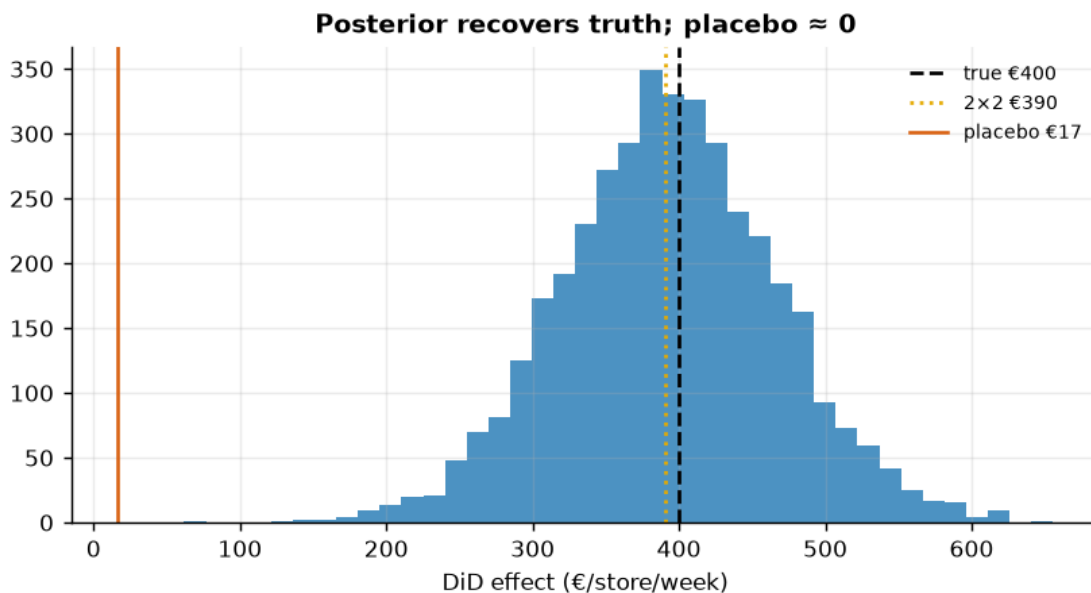
We now put a number and an interval on that jump. One implementation detail: CausalPy's `DifferenceInDifferences` expects the classic **2x2 form** — exactly one “before” and one “after” observation per unit — so we first collapse each store's 24 weekly points into a pre-launch mean and a post-launch mean. The estimate is the `group x post` interaction coefficient β_3 , i.e. the DiD.

DiD effect €393/store/week (true €400) · 90% CI [€274, €515]
 DiD convergence: max r-hat 1.000 - min ESS 1596 - divergences 0

1.4 5 · Validate — 2x2 cross-check and a placebo test

Cross-check the Bayesian estimate against the hand-computed 2x2. With revenue **standardized** before the fit (so CausalPy's default $O(1)$ -scaled priors don't shrink a €400 effect sitting on €1000 revenue), the two now **agree** and both land on the planted €400 with the truth inside the interval. Then **falsify**: run DiD on the *pre-period only*, splitting it into a fake “before/after” at week 6. There was no treatment then, so the placebo effect should be ~ 0 — a large one would mean our design manufactures effects.

Bayesian €393 · 2x2 €390 · true €400 · placebo (no real effect) €17



1.5 6 · Decide, in euros — the rollout case

Under parallel trends the DiD number is the **ATT** — **the effect on the pilot stores**. Rolling the app out to all 500 assumes the other 480 respond the same way, which chains rarely guarantee

(pilot sites aren't picked at random). So alongside the headline projection we **stress it two ways**: a **transfer discount** (what if the non-pilot stores realise only 75% or 50% of the pilot lift?) and a **cost sweep** (how high can the app's running cost climb before the call flips?). The table below is P(app pays) across both — robust at full transfer, but a *pilot-further* call under aggressive discounting and higher running costs.

Base case (APP_COST €120, full pilot lift across 500 stores):

net €273/store/week · annual value €7,097,941 [90% €3,993,808, €10,272,351]
 · P(app pays) 1.00 -> ROLL OUT
 break-even: the app clears its cost with 95% posterior probability while the running cost stays below ~€274/store/week.

P(app pays) by transfer share (rows) and running cost in €/store/week (cols):

transfer \ cost	80	120	200	300
100% (as pilot)	1.00	1.00	0.99	0.90
75%	1.00	1.00	0.96	0.46
50%	1.00	0.98	0.46	0.00

Most the roll-out can pay per store/week and still clear cost at 90% posterior confidence:

at 100% (as pilot) up to ~€300
 at 75% up to ~€225
 at 50% up to ~€150

1.6 7 · Caveats — and the staggered-adoption trap, demonstrated

Parallel trends is the whole ballgame and untestable post-launch; a diverging pre-trend kills the design (use synthetic control, nb 07). **No spillover** between nearby stores. But the subtlest trap is **staggered adoption**: when units adopt at *different* times, naive two-way fixed-effects DiD uses already-treated units as controls (“forbidden comparisons”) and can put **negative weights** on some effects (Goodman-Bacon), biasing the estimate even when every true effect is positive. We show it live:

Naive TWFE €177 vs true average post-treatment effect €400 - bias -223. Use Callaway-Sant'Anna / Sun-Abraham for staggered rollouts.

